CSP-S 2023 复赛四题题解

> 从"最好拿的部分分"一档一档推到正解 · 每档附可直接提交的代码

四题总览

题目	名称	分档	正解核心
第一题	密码锁	2 档	枚举密码 + 生成所有锁车后状态
第二题	消消乐	4 档	前缀状态 + 哈希 + 排序统计
第三题	结构体	3 档	递归计算布局 + 递归定位
第四题	种树	4 档	二分答案 + 倒序贪心判定

所有代码统一使用万能头文件 `<bits/stdc++.h>` 与标准命名空间 `using namespace std;`，可直接复制提交。

第一题 P9752 [CSP-S 2023] 密码锁

题目描述

小 Y 有一把五个拨圈的密码锁。每个拨圈上是从 0 到 9 的数字。每个拨圈都是从 0 到 9 的循环，即 9 拨动一个位置后可以变成 0 或 8。

因为校园里比较安全，小 Y 采用的锁车方式是：从正确密码开始，随机转动密码锁仅一次；每次都是以某个幅度仅转动一个拨圈，或者同时转动两个相邻的拨圈。

当小 Y 选择同时转动两个相邻拨圈时，两个拨圈转动的幅度相同。即小 Y 可以将密码锁从 `0 0 1 1 5` 转成 `1 1 1 1 5`，但不会转成 `1 2 1 1 5`。

时间久了，小 Y 也担心这么锁车的安全性，所以小 Y 记下了自己锁车后密码锁的 n 个状态，注意这 n 个状态都不是正确密码。

为了检验这么锁车的安全性，请你计算：有多少种可能的正确密码，使得它能够按照这种锁车方式产生锁车后密码锁的全部 n 个状态。

输入格式

输入的第一行包含一个正整数 n，表示锁车后密码锁的状态数。接下来 n 行每行包含五个整数，表示一个密码锁的状态。

输出格式

输出一行包含一个整数，表示这 n 个状态按照给定的锁车方式能对应多少种正确密码。

输入输出样例

输入 #1	输出 #1
`1` `0 0 1 1 5`	`81`

【样例 1 解释】 一共有 81 种可能的方案。其中转动一个拨圈的方案有 45 种，转动两个拨圈的方案有 36 种。

【样例 2】 见选手目录下的 `lock/lock2.in` 与 `lock/lock2.ans`。

数据范围

对于所有测试数据有：$1 \leqslant n \leqslant 8$。

测试点	n ⩽	特殊性质
1~3	1	无
4~5	2	无
6~8	8	A
9~10	8	无

特殊性质 A：保证所有正确密码都可以通过仅转动一个拨圈得到测试数据给出的 n 个状态。

第一档：先把 n = 1 这个最好拿的部分分吃到手（预估 30 分）

思路分析

拿到题目先看数据范围表：测试点 1~3 里 n ⩽ 1，也就是只记录了一个锁车后的状态，这三个点一共 30 分，非常好拿，先想办法吃下来。

只记录了一个状态 s 时，问题变成：有多少个密码 x，能"转动一次"变成 s?

这里有一个很好用的观察：**转动操作是可逆的**。转 d 格的逆操作就是转 10 − d 格（因为拨圈是 0~9 循环的），而 d 从 1 到 9，所以 10 − d 也在 1 到 9 之间，同样是合法的一次转动。因此

> 「x 能转动一次变成 s」等价于「s 能转动一次变成 x」。

于是我们只需要比较 x 和 s 的差别，看这个差别是不是"合法的一次转动"造成的：

- 只有一位不一样：这就是转动了一个拨圈，合法；
- 恰好两位不一样，并且这两位相邻，而且它们变化的幅度相同：这就是同时转动了两个相邻的拨圈，合法；
- 其他情况：一次转动做不到。

顺着这个判断，直接把 00000 到 99999 这 10 万个密码全试一遍，数一数有多少个满足条件就行了。答案是 81（45 + 36），和样例输出完全对上。

这一档只处理 n = 1，代码非常短。

本档代码（预估得分：30 分，对应测试点 1~3）

```
// P9752 [CSP-S 2023] 密码锁 —— 第一档：先把 n = 1 的部分分吃到手
// 预估得分：30 分（对应测试点 1~3）
#include <bits/stdc++.h>
using namespace std;

int n;
int st[10][5];          // st[i]：第 i 个记录下来的状态

// 判断：密码 x 能不能"转动一次"变成状态 s
```

```
// 操作可逆（转 d 格的逆操作是转 10 - d 格），只需比较 x 和 s 的差别
bool ok(int x[], int s[])
{
    int df[5];          // df[j]：第 j 位的变化量，0 表示这一位没变
    int cnt = 0;        // 发生变化的位数
    for (int j = 0; j < 5; j++)
    {
        df[j] = (s[j] - x[j] + 10) % 10;
        if (df[j] != 0)
        {
            cnt++;
        }
    }
    // 情况一：只有一位变了 —— 转动了一个拨圈
    if (cnt == 1)
    {
        return true;
    }
    // 情况二：恰好两位变了，这两位相邻，且变化幅度相同
    //         —— 同时转动了两个相邻的拨圈
    if (cnt == 2)
    {
        for (int j = 0; j < 4; j++)
        {
            if (df[j] != 0 && df[j] == df[j + 1])
            {
                return true;
            }
        }
    }
    return false;
}

int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < 5; j++)
        {
            scanf("%d", &st[i][j]);
        }
    }
    // n = 1 时：只要判断哪些密码能一步转到这一个状态即可
    if (n == 1)
    {
        int ans = 0;
        int p[5];
        // 枚举全部 10 万种可能的正确密码
        for (int v = 0; v < 100000; v++)
        {
            int t = v;
            for (int j = 4; j >= 0; j--)
            {
                p[j] = t % 10;
                t = t / 10;
            }
            if (ok(p, st[0]))
            {
```

```
                ans++;
            }
        }
        printf("%d\n", ans);
    }
    else
    {
        // 还不会做一般情况，这部分分拿不到
        printf("0\n");
    }
    return 0;
}
```

小结

这一档告诉我们的两件事：

1. 数据范围表里的 "n ⩽ 1" 就是送分信号，遇到不会做的题，先把这种白送的分数写稳；
2. "操作可逆"这个观察很关键，它让我们可以直接比较"密码"和"状态"的差别，而不用真的去模拟转动。

第二档：正解 —— 枚举密码，生成所有锁车后的状态（预估 100 分）

思路分析

现在处理一般的 n（最多 8）。顺着第一档的想法继续想：

- 正确密码一共就只有 00000 ~ 99999 这 10 万种，完全可以全部枚举；
- 对于每一个密码，我们把它"转动一次"能得到的所有状态都列出来；
- 如果记录下来的 n 个状态全部都在这个集合里，那么这个密码就是可能的正确密码。

每个密码能转动出多少种状态？

- 只转动一个拨圈：选哪个拨圈有 5 种，转动幅度有 9 种（1~9，转 10 格等于没转），共 $5 \times 9 = 45$ 种；
- 同时转动两个相邻拨圈：相邻的拨圈有 4 对，转动幅度 9 种，共 $4 \times 9 = 36$ 种。

合计 81 种。于是总计算量是 $10^5 \times 81 \approx 8.1 \times 10^6$，非常小，随便跑。

一个必须注意的性能坑：如果每换一个密码就把标记数组用循环清空一次，那么清空的总代价是 $10^5 \times 10^5 = 10^{10}$，一定会超时。这里用"时间戳"的小技巧：

```
cur++;              // 换一个密码，时间戳加一
tag[状态] = cur;     // 判断时用 tag[状态] == cur 即可，不用清空
```

这样不同密码之间就不会互相干扰，也完全不需要清空数组。

另外两个容易踩的坑：

- 转动幅度必须从 1 取到 9。如果允许幅度为 0，密码本身也会被算进去，而题目明确说"这 n 个状态都不是正确密码"；
- 同时转动两个相邻拨圈时，两位的转动幅度必须相同，不能一位转 1 格另一位转 2 格。

本档代码（预估得分：100 分）

```cpp
// P9752 [CSP-S 2023] 密码锁 —— 第二档：正解
// 预估得分：100 分
#include <bits/stdc++.h>
using namespace std;

int n;
int st[10][5];          // st[i]：第 i 个记录下来的状态
int rec[10];            // rec[i]：第 i 个状态对应的整数编号
int p[5];               // 当前正在枚举的正确密码
int q[5];               // 临时数组：转动之后得到的状态
int tag[100000];        // tag[v] == cur 表示：当前这个密码能产生状态 v
int cur;                // 时间戳，每换一个密码就加一

// 把五个拨圈上的数字拼成一个 0~99999 的整数，方便当数组下标使用
int num(int a[])
{
    int res = 0;
    for (int j = 0; j < 5; j++)
    {
        res = res * 10 + a[j];
    }
    return res;
}

int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < 5; j++)
        {
            scanf("%d", &st[i][j]);
        }
        rec[i] = num(st[i]);
    }
    int ans = 0;
    // 枚举每一种可能的正确密码：00000 ~ 99999，一共 10 万种
    for (int v = 0; v < 100000; v++)
    {
        // 把 v 拆成五个拨圈上的数字
        int t = v;
        for (int j = 4; j >= 0; j--)
        {
            p[j] = t % 10;
            t = t / 10;
        }
        // 换了一个密码，时间戳加一
        // 这样就不必用循环把 tag 数组清空（清空 10 万次会非常慢）
        cur++;
        // 情况一：只转动一个拨圈
        // 枚举转动第 j 个拨圈，幅度 d 从 1 到 9（转 10 格等于没转）
        for (int j = 0; j < 5; j++)
        {
            for (int d = 1; d <= 9; d++)
            {
                for (int k = 0; k < 5; k++)
```

```
                {
                    q[k] = p[k];
                }
                q[j] = (q[j] + d) % 10;          // 拨圈是 0~9 循环的
                tag[num(q)] = cur;
            }
        }
        // 情况二：同时转动两个相邻的拨圈，且转动幅度相同
        // 相邻的两个拨圈只有 4 对：(0,1) (1,2) (2,3) (3,4)
        for (int j = 0; j < 4; j++)
        {
            for (int d = 1; d <= 9; d++)
            {
                for (int k = 0; k < 5; k++)
                {
                    q[k] = p[k];
                }
                q[j] = (q[j] + d) % 10;
                q[j + 1] = (q[j + 1] + d) % 10;
                tag[num(q)] = cur;
            }
        }
        // 检查：记录下来的 n 个状态是不是都能由这个密码转动一次得到
        bool gd = true;
        for (int i = 0; i < n; i++)
        {
            if (tag[rec[i]] != cur)
            {
                gd = false;
                break;
            }
        }
        if (gd)
        {
            ans++;
        }
    }
    printf("%d\n", ans);
    return 0;
}
```

复杂度

时间复杂度 $O(10^5 \times 81)$，空间复杂度 $O(10^5)$。整份代码跑起来不到 0.1 秒。

易错点提醒

1. 拨圈是循环的，转动后一定要 `% 10`；
2. 五个拨圈的下标别搞混，建议统一用 0~4；
3. 判断"是否覆盖全部 n 个状态"时，一旦有一个状态不在集合里就要立刻跳出，别漏掉 `break`；
4. 题目只保证"这 n 个状态都不是正确密码"，但没说这 n 个状态互不相同。用标记方式处理（而不是计数方式）可以天然地避免重复状态带来的问题。

第二题 P9753 [CSP-S 2023] 消消乐

题目描述

小 L 现在在玩一个低配版本的消消乐，该版本的游戏是一维的，一次也只能消除两个相邻的元素。

现在，他有一个长度为 n 且仅由小写字母构成的字符串。我们称一个字符串是可消除的，当且仅当可以对这个字符串进行若干次操作，使之成为一个空字符串。其中每次操作可以从字符串中删除两个相邻的相同字符，操作后剩余字符串会拼接在一起。

小 L 想知道，这个字符串的所有非空连续子串中，有多少个是可消除的。

输入格式

输入的第一行包含一个正整数 n，表示字符串的长度。第二行包含一个长度为 n 且仅由小写字母构成的字符串。

输出格式

输出一行包含一个整数，表示题目询问的答案。

输入输出样例

输入 #1	输出 #1
`8` `accabccb`	`5`

【样例 1 解释】 一共有 5 个可消除的连续子串，分别是 `cc`、`acca`、`cc`、`bccb`、`accabccb`。

【样例 2】 见选手目录下的 `game/game2.in` 与 `game/game2.ans`。

数据范围

对于所有测试数据有：$1 \leqslant n \leqslant 2 \times 10^6$，且字符串仅由小写字母构成。

测试点	n ⩽	特殊性质
1~5	10	无
6~7	800	无
8~10	8000	无

测试点	n ⩽	特殊性质
11~12	2×10^5	A
13~14	2×10^5	B
15~17	2×10^5	无
18~20	2×10^6	无

特殊性质 A：字符串中的每个字符独立等概率地从字符集中选择。

特殊性质 B：字符串仅由 a 和 b 构成。

第一档：暴力枚举子串 + 模拟消除（预估 25 分）

思路分析

最朴素的想法：枚举每一个子串，判断它能不能消成空串。

怎么判断一个串能不能消成空串？用栈模拟即可：

- 从左到右扫这个串的每一个字符；
- 如果它和栈顶字符相同，说明这两个相邻相同字符可以消掉，弹出栈顶；
- 否则把它压进栈；
- 全部扫完后，栈是空的就说明可以消成空串。

这个模拟是 O(子串长度) 的。子串一共有 $O(n^2)$ 个，所以总复杂度 $O(n^3)$。当 n ⩽ 10（测试点 1~5，共 25 分）时完全够用。

本档代码（预估得分：25 分，对应测试点 1~5）

```cpp
// P9753 [CSP-S 2023] 消消乐 —— 第一档：枚举子串 + 模拟消除
// 预估得分：25 分（对应测试点 1~5，n <= 10）
#include <bits/stdc++.h>
using namespace std;

const int N = 25;

int n;
char s[N];
char stk[N];            // 模拟消除用的栈

// 判断子串 s[l..r] 能不能被消成空串
// 从左到右扫，和栈顶相同就弹出栈顶，否则压入栈；最后栈空即可
bool ok(int l, int r)
{
    int top = 0;
    for (int i = l; i <= r; i++)
    {
        if (top > 0 && stk[top - 1] == s[i])
```

```
        {
            top--;                          // 和栈顶相同，消掉一对
        }
        else
        {
            stk[top] = s[i];
            top++;
        }
    }
    return top == 0;
}

int main()
{
    scanf("%d", &n);
    scanf("%s", s);
    long long ans = 0;
    // 枚举所有子串的左右端点（下标从 0 开始）
    for (int l = 0; l < n; l++)
    {
        for (int r = l; r < n; r++)
        {
            if (ok(l, r))
            {
                ans++;
            }
        }
    }
    printf("%lld\n", ans);
    return 0;
}
```

第二档：固定左端点，向右扩展时增量维护（预估 50 分）

思路分析

第一档慢在哪？每换一个子串都要把整个子串重新扫一遍，做了大量重复计算。

换个枚举方式：固定左端点 l，让右端点 r 从 l 一直加到 n − 1，并且把栈"继承"下去。

关键观察：栈里只多加入末尾一个新字符 s[r]，那么它最多只会和栈顶消掉一对，消完这次之后不会再连锁触发新的消除（因为剩下的栈顶和它下面的字符本来就不同，否则之前就消掉了）。所以每加入一个字符只需要 $O(1)$ 的判断，不需要重扫。

于是复杂度变成 $O(n^2)$，能过 $n \leqslant 8000$ 的测试点 1~10，共 50 分。

本档代码（预估得分：50 分，对应测试点 1~10）

```
// P9753 [CSP-S 2023] 消消乐 —— 第二档：固定左端点，向右扩展时增量维护栈
// 预估得分：50 分（对应测试点 1~10，n <= 8000）
#include <bits/stdc++.h>
using namespace std;

const int N = 8005;
```

```
int n;
char s[N];
char stk[N];            // 模拟消除用的栈

int main()
{
    scanf("%d", &n);
    scanf("%s", s);
    long long ans = 0;
    // 固定左端点 l，把右端点 r 从 l 一直加到 n - 1
    for (int l = 0; l < n; l++)
    {
        int top = 0;            // 每换一个左端点，栈重新变空
        for (int r = l; r < n; r++)
        {
            // 只在末尾多加入一个字符 s[r]
            // 它最多只会和栈顶消掉一对，所以一次判断就够了
            if (top > 0 && stk[top - 1] == s[r])
            {
                top--;
            }
            else
            {
                stk[top] = s[r];
                top++;
            }
            // 栈空说明 s[l..r] 可以被消成空串
            if (top == 0)
            {
                ans++;
            }
        }
    }
    printf("%lld\n", ans);
    return 0;
}
```

第三档：只有 a 和 b 时，可以用 O(n) 做完（预估 60 分）

思路分析

这一档对付特殊性质 B：字符串里只有 a 和 b 两种字符（测试点 13~14，n 可以到 2×10^5，$O(n^2)$ 肯定不行）。

先介绍一个本题最关键的观察（正解也靠它）。设 f(i) 表示"前 i 个字符消完之后剩下的那个串"。那么

> 子串 s[l..r] 可以消成空串 $\Longleftrightarrow$ f(l − 1) = f(r)。

为什么？因为消除的过程具有"结合性"：把前 l − 1 个字符消完得到 f(l − 1)，接着读入 s[l..r] 继续消；如果 s[l..r] 能自己消成空串，那么剩下的就还是 f(l − 1)，也就是 f(r) = f(l − 1)。反过来，如果 f(r) = f(l − 1)，说明中间这段 s[l..r] 对最终结果没有任何贡献，它自己正好被消干净了。

所以问题变成：在 f(0), f(1), ..., f(n) 这 n + 1 个"前缀消完后的串"里，有多少对是相同的。只要有 cnt 个前缀的 f 值相同，就能配出 cnt(cnt − 1) / 2 个可消除子串。

接下来的问题是：怎么表示和比较 f(i)？只有 a 和 b 时有一个特别好的性质：栈里相邻两个字符一定不同（相同就消掉了），所以栈的内容一定是 a、b 交替的。交替串只要知道长度和栈底那个字符，整个串就唯一确定了！

于是每个前缀状态用 (长度, 栈底字符) 这两个值就能表示，比较是 O(1) 的。扫一遍字符串，边维护栈边用桶统计每种状态出现几次：

- 当前前缀状态和之前某个前缀相同，就又能配出一个可消除子串；
- 所以扫到第 i 个前缀时，把"之前有多少个前缀和自己状态相同"累加进答案即可。

整个过程 O(n)，非常快。

说明：下面这份代码里，`slvAB` 是只有 a、b 时的 O(n) 做法，`slv2` 是上一档的 $O(n^2)$ 兜底。程序开头判断一下串里是不是只有 a 和 b，是就走 `slvAB`，否则走 `slv2`。这样一份代码能同时拿下测试点 1~10（50 分）和 13~14（10 分），合计 60 分。

本档代码（预估得分：60 分，对应测试点 1~10 与 13~14）

```
// P9753 [CSP-S 2023] 消消乐 —— 第三档：只有 a 和 b 时的 O(n) 做法
// 预估得分：60 分（测试点 1~10 由 slv2 兜底，13~14 由 slvAB 拿下）
#include <bits/stdc++.h>
using namespace std;

const int N = 200005;

int n;
char s[N];
char stk[N];                // 兜底做法里模拟消除用的栈
long long cnt[N][2];        // cnt[k][c]：状态（长度 k、栈底类别 c）出现次数

// 只有 a、b 时的 O(n) 做法
// 栈里一定是 a、b 交替的，"长度 + 栈底字符"就能唯一确定栈的内容
void slvAB()
{
    int top = 0;            // 栈里还剩几个字符
    char bot = 0;           // 栈底那个字符（栈空时不看它）
    long long ans = 0;
    cnt[0][0] = 1;          // 空前缀：长度 0 的空栈，先记一次
    for (int i = 1; i <= n; i++)
    {
        char c = s[i];
        // 算出当前栈顶字符（栈为空时给一个不可能出现的 0）
        char tc = 0;
        if (top > 0)
        {
            if (top % 2 == 1)
            {
                tc = bot;               // 长度为奇数，栈顶就是栈底字符
            }
            else
            {
                tc = 'a' + 'b' - bot;   // 长度为偶数，栈顶是另一个字符
```

```
            }
        }
        if (tc == c)
        {
            top--;                          // 和栈顶相同，消掉一对
        }
        else
        {
            if (top == 0)
            {
                bot = c;                    // 空栈压入，它就是新的栈底
            }
            top++;
        }
        // 之前出现过多少个相同的前缀状态，就多了多少个可消除子串
        int cb = 0;
        if (top > 0 && bot == 'b')
        {
            cb = 1;
        }
        ans = ans + cnt[top][cb];
        cnt[top][cb]++;
    }
    printf("%lld\n", ans);
}

// 兜底：第二档的 O(n^2) 增量做法
void slv2()
{
    long long ans = 0;
    for (int l = 1; l <= n; l++)
    {
        int top = 0;
        for (int r = l; r <= n; r++)
        {
            if (top > 0 && stk[top - 1] == s[r])
            {
                top--;
            }
            else
            {
                stk[top] = s[r];
                top++;
            }
            if (top == 0)
            {
                ans++;
            }
        }
    }
    printf("%lld\n", ans);
}

int main()
{
    scanf("%d", &n);
    scanf("%s", s + 1);          // 下标从 1 开始
    // 判断这个串是不是只有 a 和 b
    bool ab = true;
```

```
    for (int i = 1; i <= n; i++)
    {
        if (s[i] != 'a' && s[i] != 'b')
        {
            ab = false;
        }
    }
    if (ab)
    {
        slvAB();
    }
    else
    {
        slv2();
    }
    return 0;
}
```

第四档：正解 —— 前缀状态 + 哈希 + 排序统计（预估 100 分）

思路分析

n 到了 2×10^6，$O(n^2)$ 也不行了。上一档我们已经把问题转化成了"统计相同的前缀状态"，但那个 O(n) 做法只在 a、b 两种字符时成立——字母一多，栈的内容就不再是交替串，没法只用"长度 + 栈底字符"表示。

一般字符集下，怎么表示和比较 f(i) 这个"消完剩下的串"？直接存字符串肯定不行（总长可能到 $O(n^2)$）。用哈希：

- 用一个栈维护当前没被消掉的字符，同时维护"栈深为 top 时的哈希值" H(top)；
- 压入字符 c 时：$H(top) = H(top-1) \times base + c$；
- 弹出时直接 top 减一，上一层的哈希值本来就记在数组里，直接读就行。

这样每个前缀的 f(i) 都压缩成了一个哈希值，扫一遍字符串就全部算完，O(n)。

最后把 n + 1 个哈希值排序（排序规则自己写成子函数），相同的值会挨在一起，数一数每一段有多长，累加 $cnt(cnt-1)/2$ 就是答案。总复杂度 O(n log n)。

关于哈希冲突：单个哈希有极小概率把两个不同的串算成同一个值。为了保险，代码里用了两个不同的哈希（一个是 unsigned long long 自然溢出，另一个是取模 $10^9 + 7$，底数也不同），只有两个值都相同才认为是同一个串，冲突概率低到可以忽略。

本档代码（预估得分：100 分）

```
// P9753 [CSP-S 2023] 消消乐 —— 第四档：正解（前缀状态哈希 + 排序统计）
// 预估得分：100 分
#include <bits/stdc++.h>
using namespace std;

const int N = 2000005;
const unsigned long long B1 = 131ULL;   // 第一个哈希的底数（自然溢出）
const long long B2 = 13131LL;           // 第二个哈希的底数（取模）
const long long P2 = 1000000007LL;
```

```cpp
// 一个前缀"消完之后剩下的样子"，用一对哈希值来表示
struct Key
{
    unsigned long long h1;
    long long h2;
};

int n;
char s[N];
char stc[N];                // 栈里存的是没被消掉的字符
unsigned long long h1[N];   // h1[t]：栈深为 t 时的第一个哈希值
long long h2[N];            // h2[t]：栈深为 t 时的第二个哈希值
Key key[N];                 // key[i]：前 i 个字符消完之后的哈希值

// 排序规则：先按 h1 比，h1 相同再按 h2 比
bool cmp(const Key &a, const Key &b)
{
    if (a.h1 != b.h1)
    {
        return a.h1 < b.h1;
    }
    return a.h2 < b.h2;
}

// 判断两个前缀状态是否相同
bool same(const Key &a, const Key &b)
{
    return a.h1 == b.h1 && a.h2 == b.h2;
}

int main()
{
    scanf("%d", &n);
    scanf("%s", s + 1);             // 让下标从 1 开始
    int top = 0;                    // 栈顶位置，top == 0 表示栈空
    h1[0] = 0;
    h2[0] = 0;
    key[0].h1 = 0;                  // 空前缀消完之后还是空串
    key[0].h2 = 0;
    for (int i = 1; i <= n; i++)
    {
        char c = s[i];
        if (top > 0 && stc[top] == c)
        {
            // 和栈顶相同，消掉一对：栈回退到上一步的状态
            top--;
        }
        else
        {
            // 压入栈，并在上一个状态的基础上算出新的哈希值
            top++;
            stc[top] = c;
            h1[top] = h1[top - 1] * B1 + (unsigned long long)c;
            h2[top] = (h2[top - 1] * B2 + (long long)c) % P2;
        }
        key[i].h1 = h1[top];
        key[i].h2 = h2[top];
    }
```

```
    // 把 n + 1 个前缀状态排序，相同的状态会排到一起
    sort(key, key + n + 1, cmp);
    // 统计每种状态出现次数，从 cnt 个里任选 2 个就是一个可消除子串
    long long ans = 0;
    long long cnt = 1;
    for (int i = 1; i <= n; i++)
    {
        if (same(key[i], key[i - 1]))
        {
            cnt++;
        }
        else
        {
            ans = ans + cnt * (cnt - 1) / 2;
            cnt = 1;
        }
    }
    ans = ans + cnt * (cnt - 1) / 2;
    printf("%lld\n", ans);
    return 0;
}
```

复杂度

时间复杂度 O(n log n)（瓶颈在排序），空间复杂度 O(n)。n = 2 × 10^6 时运行时间在 1 秒以内。

易错点提醒

1. 答案可能达到 n(n + 1) / 2 ≈ 2 × 10^{12}，一定要用 `long long`；
2. 前缀状态一共是 n + 1 个（包括"一个字符都没有"的空前缀 f(0)），排序和统计时别漏掉它；
3. 弹出栈顶时不要重新计算哈希，直接回退到上一层记录的哈希值，这是能在 O(n) 内完成的关键；
4. 判断两个状态是否相同时，两个哈希都要比。

第三题 P9754 [CSP-S 2023] 结构体

题目描述

在 C++ 等高级语言中，除了 int 和 float 等基本类型外，通常还可以自定义结构体类型。在本题当中，你需要模拟一种类似 C++ 的高级语言的结构体定义方式，并计算出相应的内存占用等信息。

在这种语言中，基本类型共有 4 种：`byte`、`short`、`int`、`long`，分别占据 1、2、4、8 字节的空间。

定义一个结构体类型时，需要给出类型名和成员，其中每个成员需要按顺序给出类型和名称。类型可以为基本类型，也可以为先前定义过的结构体类型。注意，定义结构体类型时不会定义具体元素，即不占用内存。

定义一个元素时，需要给出元素的类型和名称。元素将按照以下规则占据内存：

- 元素内的所有成员将按照定义时给出的顺序在内存中排布，对于类型为结构体的成员同理。
- 为了保证内存访问的效率，元素的地址占用需要满足对齐规则，即任何类型的大小和该类型元素在内存中的起始地址均应对齐到该类型对齐要求的整数倍。具体而言：
- 对于基本类型：对齐要求等于其占据空间大小，如 int 类型需要对齐到 4 字节，其余同理。
- 对于结构体类型：对齐要求等于其成员的对齐要求的最大值，如一个含有 int 和 short 的结构体类型需要对齐到 4 字节。

以下是一个例子（以 C++ 语言的格式书写）：

```
struct d {
    short a;
    int b;
    short c;
};
d e;
```

该代码定义了结构体类型 d 与元素 e。元素 e 包含三个成员 e.a、e.b、e.c，分别占据第 0~1、4~7、8~9 字节的地址。由于类型 d 需要对齐到 4 字节，因此 e 占据了第 0~11 字节的地址，大小为 12 字节。

你需要处理 n 次操作，每次操作为以下四种之一：

1. 定义一个结构体类型。给定正整数 k 与字符串 s, t_1, n_1, ..., t_k, n_k，其中 k 表示该类型的成员数量，s 表示类型名，t_1, t_2, ..., t_k 按顺序分别表示每个成员的类型，n_1, n_2, ..., n_k 分别表示每个成员的名称。你需要输出该结构体类型的大小和对齐要求，用一个空格分隔。
2. 定义一个元素。给定字符串 t, n 分别表示该元素的类型与名称。所有被定义的元素将按顺序，从内存地址为 0 开始依次排开，并需要满足地址对齐规则。你需要输出新定义的元素的起始地址。
3. 访问某个元素。给定字符串 s，表示所访问的元素。与 C++ 等语言相同，采用 `.` 来访问结构体类型的成员。如 `a.b.c`，表示 a 是一个已定义的元素，它是一个结构体类型，有一个名称为 b 的成员，它也是一个结构体类型，有一个名称为 c 的成员。你需要输出如上被访问的最内层元素的起始地址。
4. 访问某个内存地址。给定非负整数 addr，你需要判断是否存在一个基本类型的元素占据了该地址。若是，则按操作 3 中的访问元素格式输出该元素；否则输出 `ERR`。

输入格式

第 1 行：一个正整数 n，表示操作的数量。接下来若干行，依次描述每个操作，每行第一个正整数 op 表示操作类型：

- 若 op = 1，首先输入一个字符串 s 与一个正整数 k，接下来 k 行每行输入两个字符串 t_i, n_i；
- 若 op = 2，输入两个字符串 t, n；
- 若 op = 3，输入一个字符串 s；
- 若 op = 4，输入一个非负整数 addr。

输出格式

输出 n 行，依次表示每个操作的输出结果，输出要求如题目描述中所述。

输入输出样例

输入 #1	输出 #1

`5`
`1 a 2`
`short aa`
`int ab`
`1 b 2`
`a ba`
`long bb`
`2 b x`
`3 x.ba.ab`
`4 10`
	`8 4`
	`16 8`
	`0`
	`4`
	`x.bb`

【样例 1 解释】 结构体类型 a 中，short 类型的成员 aa 占据第 0~1 字节地址，int 类型的成员 ab 占据第 4~7 字节地址。又由于其对齐要求为 4 字节，可得其大小为 8 字节。由此可同理计算出结构体类型 b 的大小为 16 字节，对齐要求为 8 字节。

【样例 2】 见选手目录下的 `struct/struct2.in` 与 `struct/struct2.ans`。**【样例 2 解释】** 第二个操作 4 中，访问的内存地址恰好在为了地址对齐而留下的"洞"里，因此没有基本类型元素占据它。

数据范围

对于全部数据，满足 $1 \leqslant n \leqslant 100$，$1 \leqslant k \leqslant 100$，$0 \leqslant addr \leqslant 10^{18}$。所有名称由不超过 10 个字符的小写字母组成，且都不是 `byte`，`short`，`int`，`long`。所有定义的结构体类型名和元素名称互不相同，同一结构体内成员名称互不相同。保证所有操作均合法，任意结构体大小及定义的元素占据的最高内存地址均不超过 10^{18}。

测试点	特殊性质
1	A、D
2~3	A
4~5	B、D
6~8	B
9~10	C、D
11~13	C
14~16	D
17~20	无

特殊性质 A：没有操作 1。

特殊性质 B：只有一个操作 1。

特殊性质 C：所有操作 1 中给出的成员类型均为基本类型。

特殊性质 D：基本类型只有 long。

提示（对齐和大小的形式化定义）

对于结构体类型的对齐要求和大小：

- 设该结构体内有 k 个成员，其大小分别为 $s_1, \ldots, s_k$，对齐要求分别为 $a_1, \ldots, a_k$；
- 则该结构体的对齐要求为 $a = \max\{a_1, \ldots, a_k\}$；
- 再设这些成员排布时的地址偏移量分别为 $o_1, \ldots, o_k$，则：$o_1 = 0$；对于 $i = 2, \ldots, k$，o_i 为满足 $o_{i-1} + s_{i-1} \leqslant o_i$ 且 a_i 整除 o_i 的最小值；
- 则该结构体的大小 s 为满足 $o_k + s_k \leqslant s$ 且 a 整除 s 的最小值。

对于定义元素时的内存排布：

- 设第 i 个被定义的元素大小为 s_i，对齐要求为 a_i，起始地址为 b_i；
- 则 $b_1 = 0$，对于 $i \geqslant 2$，b_i 为满足 $b_{i-1} + s_{i-1} \leqslant b_i$ 且 a_i 整除 b_i 的最小值。

第一档：没有结构体，只有基本类型（预估 15 分）

思路分析

这是一道"大模拟"题：题目本身不难，但细节多、代码长，考的是能不能把规则一条一条落实清楚。这种题最怕一上来就想写完整版，写完发现调不出来。正确做法是先从特殊性质入手，一步一步加功能。

看特殊性质 A：没有操作 1，也就是根本不会出现结构体类型。这样：

- 类型只有四种基本类型，它们的大小 = 对齐要求 = 1, 2, 4, 8；
- 操作 2：定义元素，起始地址就是把当前内存末尾向上取整到"对齐要求"的倍数；
- 操作 3：访问的元素就是元素自己（没有 `.`），直接输出起始地址；
- 操作 4：扫一遍所有元素，看这个地址落在哪个元素的区间 [起始地址, 起始地址 + 大小) 里，输出它的名字；找不到就输出 `ERR`。

这一档对应测试点 1~3，一共 15 分，代码很短，几分钟就能写完，非常稳。

"向上取整到 a 的倍数"这个操作后面每一档都要用，单独写成一个函数：

```
long long rup(long long x, long long a)
{
    if (x % a == 0)
    {
        return x;
    }
    return (x / a + 1) * a;
}
```

本档代码（预估得分：15 分，对应测试点 1~3）

```
// P9754 [CSP-S 2023] 结构体 —— 第一档：只有基本类型（没有操作 1）
// 预估得分：15 分（对应特殊性质 A 的测试点 1~3）
#include <bits/stdc++.h>
using namespace std;

long long bsz[4] = {1, 2, 4, 8};    // byte short int long 的大小
                                    // 基本类型的对齐要求就等于它的大小

// 一个元素（真正占内存的东西）
struct Elem
{
    char nm[15];
    long long sz;                   // 占多少字节
    long long at;                   // 起始地址
};

Elem el[105];
int ne;                             // 已经定义的元素个数
long long ed;                       // 当前已经用到的内存的下一个地址

// 把地址 x 向上取整到 a 的倍数
long long rup(long long x, long long a)
{
    if (x % a == 0)
    {
        return x;
    }
    return (x / a + 1) * a;
}

// 按名字找基本类型的编号
```

```
int gt(char s[])
{
    if (strcmp(s, "byte") == 0)
    {
        return 0;
    }
    if (strcmp(s, "short") == 0)
    {
        return 1;
    }
    if (strcmp(s, "int") == 0)
    {
        return 2;
    }
    return 3;                       // "long"
}

// 按名字找元素编号
int ge(char s[])
{
    for (int i = 0; i < ne; i++)
    {
        if (strcmp(el[i].nm, s) == 0)
        {
            return i;
        }
    }
    return -1;
}

int main()
{
    ne = 0;
    ed = 0;
    int n;
    scanf("%d", &n);
    for (int qi = 0; qi < n; qi++)
    {
        int op;
        scanf("%d", &op);
        if (op == 2)
        {
            // 定义一个基本类型的元素
            char tn[15], en[15];
            scanf("%s%s", tn, en);
            int tid = gt(tn);
            long long sz = bsz[tid];
            long long st = rup(ed, sz);         // 起始地址要对齐
            el[ne].sz = sz;
            el[ne].at = st;
            strcpy(el[ne].nm, en);
            ne++;
            ed = st + sz;
            printf("%lld\n", st);
        }
        else if (op == 3)
        {
            // 没有结构体，访问的就是元素自己
            char s[15];
```

```
            scanf("%s", s);
            int id = ge(s);
            printf("%lld\n", el[id].at);
        }
        else if (op == 4)
        {
            // 判断这个地址被哪个元素占着
            long long x;
            scanf("%lld", &x);
            int id = -1;
            for (int i = 0; i < ne; i++)
            {
                if (x >= el[i].at && x < el[i].at + el[i].sz)
                {
                    id = i;
                    break;
                }
            }
            if (id == -1)
            {
                printf("ERR\n");                // 这个地址不属于任何元素
            }
            else
            {
                printf("%s\n", el[id].nm);
            }
        }
    }
    return 0;
}
```

第二档：结构体的成员都是基本类型（预估 65 分）

思路分析

现在加上结构体，但先做特殊性质 C 的情形：操作 1 里给出的成员类型都是基本类型，也就是结构体只有一层，不会出现"结构体套结构体"。

多出来的两件事：

1. 操作 1（定义结构体类型）：按成员给出的顺序，一个一个往后摆。第 i 个成员要先对齐（把当前偏移量向上取整到它对齐要求的倍数），摆完后偏移量加上它的大小。同时记下所有成员对齐要求的最大值 ma。全部摆完后，偏移量还要再向上取整到 ma 的倍数，才是这个结构体的大小；对齐要求就是 ma。
2. 操作 3 / 操作 4 要多走一层：* 操作 3 的路径形如 `x.ab`：先按前一段找到元素 x，再在 x 的类型里按名字找到成员，地址加上这个成员的偏移量；* 操作 4 找到元素后，如果元素类型是结构体，就在它的成员里找这个地址落在哪个成员上（成员都是基本类型，找到即命中），输出 `元素名.成员名`；如果一个成员都没落进去，说明这个地址落在对齐留下的"洞"里，输出 `ERR`。

这一档覆盖的测试点比看上去更多：

- 测试点 1~3（特殊性质 A，没有结构体）自然也能处理；
- 测试点 9~13（特殊性质 C，成员全是基本类型）是它的主战场；

· 测试点 4~8（特殊性质 B，只有一个操作 1）同样能过——因为只有一个操作 1 时，定义这个结构体之前根本不存在别的结构体类型，它的成员也只能是基本类型。

合起来是测试点 1~13，一共 65 分。

本档代码（预估得分：65 分，对应测试点 1~13）

```
// P9754 [CSP-S 2023] 结构体 —— 第二档：结构体的成员都是基本类型（只有一层）
// 预估得分：65 分（对应测试点 1~13）
#include <bits/stdc++.h>
using namespace std;

long long bsz[4] = {1, 2, 4, 8};    // byte short int long

// 一个结构体类型（这一档里，它的成员一定都是基本类型）
struct Type
{
    char nm[15];
    long long sz;                   // 大小
    long long al;                   // 对齐要求
    int mc;                         // 成员个数
    int mt[105];                    // 每个成员的类型编号（这里一定是 0~3）
    long long mo[105];              // 每个成员的偏移量
    char mn[105][15];               // 每个成员的名字
};

Type tp[110];                       // 0~3 是四个基本类型，之后是结构体类型
int nt;                             // 已经有的类型个数

struct Elem
{
    char nm[15];
    int tt;                         // 类型编号
    long long at;                   // 起始地址
};

Elem el[105];
int ne;
long long ed;

// 把地址 x 向上取整到 a 的倍数
long long rup(long long x, long long a)
{
    if (x % a == 0)
    {
        return x;
    }
    return (x / a + 1) * a;
}

// 按名字找类型编号
int gt(char s[])
{
    for (int i = 0; i < nt; i++)
    {
        if (strcmp(tp[i].nm, s) == 0)
        {
```

```
            return i;
        }
    }
    return -1;
}

// 按名字找元素编号
int ge(char s[])
{
    for (int i = 0; i < ne; i++)
    {
        if (strcmp(el[i].nm, s) == 0)
        {
            return i;
        }
    }
    return -1;
}

// 在类型 t 里按名字找成员编号
int gm(int t, char s[])
{
    for (int i = 0; i < tp[t].mc; i++)
    {
        if (strcmp(tp[t].mn[i], s) == 0)
        {
            return i;
        }
    }
    return -1;
}

int main()
{
    // 先把四个基本类型准备好，编号 0~3，它们都没有成员
    char bn[4][6] = {"byte", "short", "int", "long"};
    for (int i = 0; i < 4; i++)
    {
        strcpy(tp[i].nm, bn[i]);
        tp[i].sz = bsz[i];
        tp[i].al = bsz[i];
        tp[i].mc = 0;
    }
    nt = 4;
    ne = 0;
    ed = 0;
    int n;
    scanf("%d", &n);
    for (int qi = 0; qi < n; qi++)
    {
        int op;
        scanf("%d", &op);
        if (op == 1)
        {
            // 定义结构体类型（成员都是基本类型）
            char s[15];
            int k;
            scanf("%s%d", s, &k);
            int id = nt;
```

```
            nt++;
            strcpy(tp[id].nm, s);
            tp[id].mc = k;
            long long off = 0;              // 当前成员的偏移量
            long long ma = 1;               // 所有成员对齐要求的最大值
            for (int i = 0; i < k; i++)
            {
                char tn[15], mn[15];
                scanf("%s%s", tn, mn);
                int tid = gt(tn);
                long long a = bsz[tid];     // 基本类型的对齐要求等于大小
                off = rup(off, a);          // 这个成员要先对齐
                tp[id].mt[i] = tid;
                tp[id].mo[i] = off;
                strcpy(tp[id].mn[i], mn);
                off = off + bsz[tid];
                if (a > ma)
                {
                    ma = a;
                }
            }
            tp[id].al = ma;
            tp[id].sz = rup(off, ma);       // 结构体整体大小也要补齐
            printf("%lld %lld\n", tp[id].sz, tp[id].al);
        }
        else if (op == 2)
        {
            // 定义一个元素
            char tn[15], en[15];
            scanf("%s%s", tn, en);
            int tid = gt(tn);
            long long st = rup(ed, tp[tid].al);     // 起始地址要对齐
            el[ne].tt = tid;
            el[ne].at = st;
            strcpy(el[ne].nm, en);
            ne++;
            ed = st + tp[tid].sz;
            printf("%lld\n", st);
        }
        else if (op == 3)
        {
            // 这一档里路径最多两段，形如 x.ab
            char s[35];
            scanf("%s", s);
            // 按 '.' 拆成两段
            char fn[15], sn[15];
            int len = strlen(s);
            int pos = -1;
            for (int i = 0; i < len; i++)
            {
                if (s[i] == '.')
                {
                    pos = i;
                    break;
                }
            }
            if (pos == -1)
            {
                strcpy(fn, s);              // 没有 '.'，访问的就是元素自己
```

```
            sn[0] = '\0';
        }
        else
        {
            for (int i = 0; i < pos; i++)
            {
                fn[i] = s[i];
            }
            fn[pos] = '\0';
            for (int i = pos + 1; i < len; i++)
            {
                sn[i - pos - 1] = s[i];
            }
            sn[len - pos - 1] = '\0';
        }
        int id = ge(fn);
        long long addr = el[id].at;
        int ct = el[id].tt;
        if (pos != -1)
        {
            int mid = gm(ct, sn);
            addr = addr + tp[ct].mo[mid];
        }
        printf("%lld\n", addr);
    }
    else
    {
        // 操作 4：判断这个地址落在哪个基本类型成员上
        long long x;
        scanf("%lld", &x);
        // 先找落在哪个元素里
        int id = -1;
        for (int i = 0; i < ne; i++)
        {
            long long st = el[i].at;
            long long ln = tp[el[i].tt].sz;
            if (x >= st && x < st + ln)
            {
                id = i;
                break;
            }
        }
        if (id == -1)
        {
            printf("ERR\n");
        }
        else if (el[id].tt < 4)
        {
            printf("%s\n", el[id].nm);      // 元素本身就是基本类型
        }
        else
        {
            // 元素的类型是结构体，在它的成员里找（成员都是基本类型）
            int t = el[id].tt;
            long long base = el[id].at;
            int hit = -1;
            for (int i = 0; i < tp[t].mc; i++)
            {
                long long st = base + tp[t].mo[i];
```

```
                    long long ln = bsz[tp[t].mt[i]];
                    if (x >= st && x < st + ln)
                    {
                        hit = i;
                        break;
                    }
                }
                if (hit == -1)
                {
                    printf("ERR\n");            // 落在对齐留下的空洞里
                }
                else
                {
                    printf("%s.%s\n", el[id].nm, tp[t].mn[hit]);
                }
            }
        }
    }
    return 0;
}
```

第三档：正解 —— 支持任意层嵌套（预估 100 分）

思路分析

最后放开限制：结构体的成员可以是另一个结构体，可以任意层嵌套。

其实只要把第二档里的"一层"改成"递归"，代码结构完全不用变：

1. **类型的表示**：给每个类型一个编号，0~3 是四个基本类型，之后每定义一个结构体就分配一个新编号。用 `mc == 0` 来区分"基本类型"和"结构体类型"。每个类型记录：名字、大小、对齐要求、成员个数、每个成员的（类型编号、偏移量、名字）。计算结构体布局时，成员的大小和对齐要求都从类型表里查（第二档里是从基本类型表里查），这样嵌套多少层都一样处理。
2. **操作 3（访问元素）**：把 `x.ba.ab` 按 `.` 拆成若干段。第一段是元素名，之后每一段都是成员名，从外到内一层层往下走，每走一层就把地址加上对应成员的偏移量、并把当前类型换成这个成员的类型。走完输出地址。
3. **操作 4（访问地址）**：先找到地址落在哪个元素里，然后写一个递归函数 `find(类型, 起始地址)`：* 如果当前类型是基本类型（`mc == 0`），说明正好命中，直接返回；* 否则遍历它的成员，看地址落在哪个成员的区间里，把这一层的名字记下来，再递归进这个成员的类型；* 如果一个成员都没落进去，说明落在"洞"里，标记失败，最后输出 `ERR`。

几个实现细节：

- 路径最多可能有 100 多层，拆段用的数组要开够（代码里开了 210 段）；
- 地址最大到 10^{18}，所有地址、大小、偏移量都要用 `long long`；
- 找类型、找元素、找成员都写成独立的函数，主流程会非常清晰。

本档代码（预估得分：100 分）

```cpp
// P9754 [CSP-S 2023] 结构体 —— 第三档：正解（支持任意层嵌套的结构体）
// 预估得分：100 分
#include <bits/stdc++.h>
using namespace std;

// 一个"类型"：可以是基本类型，也可以是结构体类型
// 基本类型的 mc == 0，结构体类型的 mc >= 1
struct Type
{
    char nm[15];                    // 类型名
    long long sz;                   // 大小
    long long al;                   // 对齐要求
    int mc;                         // 成员个数
    int mt[105];                    // 每个成员的类型编号
    long long mo[105];              // 每个成员在结构体内的地址偏移
    char mn[105][15];               // 每个成员的名字
};

Type tp[110];                       // 0~3 是四个基本类型，之后每定义一个结构体就加一
int nt;                             // 已经有的类型个数

// 一个"元素"（真正占内存的东西）
struct Elem
{
    char nm[15];
    int tt;                         // 类型编号
    long long at;                   // 起始地址
};

Elem el[105];
int ne;
long long ed;                       // 当前已经用到的内存的下一个地址

// 把地址 x 向上取整到 a 的倍数
long long rup(long long x, long long a)
{
    if (x % a == 0)
    {
        return x;
    }
    return (x / a + 1) * a;
}

// 按名字找类型编号
int gt(char s[])
{
    for (int i = 0; i < nt; i++)
    {
        if (strcmp(tp[i].nm, s) == 0)
        {
            return i;
        }
    }
    return -1;
}
```

```
// 按名字找元素编号
int ge(char s[])
{
    for (int i = 0; i < ne; i++)
    {
        if (strcmp(el[i].nm, s) == 0)
        {
            return i;
        }
    }
    return -1;
}

// 在类型 t 里按名字找成员编号
int gm(int t, char s[])
{
    for (int i = 0; i < tp[t].mc; i++)
    {
        if (strcmp(tp[t].mn[i], s) == 0)
        {
            return i;
        }
    }
    return -1;
}

// 操作 4 需要用到的全局变量
long long qx;                       // 要查询的地址
int fd;                             // 是否找到了一个基本类型成员
char pth[210][15];                  // 从外到内记下每一层的名字
int pc;                             // 已经记了几层

// 在"类型 t、起始地址 base"这块内存里，找 qx 落在哪个基本类型成员上
// 找到就一路记下名字；落在对齐留下的空洞里就把 fd 置为 0
void find(int t, long long base)
{
    if (tp[t].mc == 0)
    {
        return;                     // 已经是基本类型，说明正好命中
    }
    for (int i = 0; i < tp[t].mc; i++)
    {
        long long st = base + tp[t].mo[i];      // 这个成员的起始地址
        long long ln = tp[tp[t].mt[i]].sz;      // 这个成员占多少字节
        if (qx >= st && qx < st + ln)
        {
            strcpy(pth[pc], tp[t].mn[i]);
            pc++;
            find(tp[t].mt[i], st);
            return;
        }
    }
    fd = 0;                         // 不在任何一个成员里，说明落在空洞中
}

int main()
{
    // 先把四个基本类型准备好，编号 0~3，它们都没有成员
    char bn[4][6] = {"byte", "short", "int", "long"};
```

```
    long long bsz[4] = {1, 2, 4, 8};
    for (int i = 0; i < 4; i++)
    {
        strcpy(tp[i].nm, bn[i]);
        tp[i].sz = bsz[i];
        tp[i].al = bsz[i];
        tp[i].mc = 0;
    }
    nt = 4;
    ne = 0;
    ed = 0;
    int n;
    scanf("%d", &n);
    for (int qi = 0; qi < n; qi++)
    {
        int op;
        scanf("%d", &op);
        if (op == 1)
        {
            // 定义一个结构体类型（成员可以是基本类型，也可以是结构体）
            char s[15];
            int k;
            scanf("%s%d", s, &k);
            int id = nt;
            nt++;
            strcpy(tp[id].nm, s);
            tp[id].mc = k;
            long long off = 0;              // 当前成员的偏移量
            long long ma = 1;               // 所有成员对齐要求的最大值
            for (int i = 0; i < k; i++)
            {
                char tn[15], mn[15];
                scanf("%s%s", tn, mn);
                int tid = gt(tn);
                long long a = tp[tid].al;
                off = rup(off, a);          // 这个成员要对齐到 a 的倍数
                tp[id].mt[i] = tid;
                tp[id].mo[i] = off;
                strcpy(tp[id].mn[i], mn);
                off = off + tp[tid].sz;
                if (a > ma)
                {
                    ma = a;
                }
            }
            tp[id].al = ma;
            tp[id].sz = rup(off, ma);       // 结构体整体大小也要补齐
            printf("%lld %lld\n", tp[id].sz, tp[id].al);
        }
        else if (op == 2)
        {
            // 定义一个元素
            char tn[15], en[15];
            scanf("%s%s", tn, en);
            int tid = gt(tn);
            long long st = rup(ed, tp[tid].al);     // 元素的起始地址也要对齐
            el[ne].tt = tid;
            el[ne].at = st;
            strcpy(el[ne].nm, en);
```

```
            ne++;
            ed = st + tp[tid].sz;
            printf("%lld\n", st);
        }
        else if (op == 3)
        {
            // 访问某个元素，形如 x.ba.ab
            char s[2005];
            scanf("%s", s);
            // 第一步：按 '.' 把这一串名字拆开
            char seg[210][15];
            int sc = 0;
            int len = strlen(s);
            int last = 0;
            for (int i = 0; i <= len; i++)
            {
                if (i == len || s[i] == '.')
                {
                    for (int j = last; j < i; j++)
                    {
                        seg[sc][j - last] = s[j];
                    }
                    seg[sc][i - last] = '\0';
                    sc++;
                    last = i + 1;
                }
            }
            // 第二步：第一段是元素名，后面每段都是成员名，一层一层往下走
            int id = ge(seg[0]);
            long long addr = el[id].at;
            int ct = el[id].tt;
            for (int i = 1; i < sc; i++)
            {
                int mid = gm(ct, seg[i]);
                addr = addr + tp[ct].mo[mid];
                ct = tp[ct].mt[mid];
            }
            printf("%lld\n", addr);
        }
        else
        {
            // 操作 4：访问某个内存地址
            scanf("%lld", &qx);
            // 先找到这个地址落在哪个元素里
            int id = -1;
            for (int i = 0; i < ne; i++)
            {
                long long st = el[i].at;
                long long ln = tp[el[i].tt].sz;
                if (qx >= st && qx < st + ln)
                {
                    id = i;
                    break;
                }
            }
            if (id == -1)
            {
                printf("ERR\n");            // 不属于任何元素
            }
```

```
            else
            {
                fd = 1;
                pc = 0;
                strcpy(pth[pc], el[id].nm);
                pc++;
                find(el[id].tt, el[id].at);
                if (fd == 0)
                {
                    printf("ERR\n");        // 落在空洞里
                }
                else
                {
                    for (int i = 0; i < pc; i++)
                    {
                        if (i > 0)
                        {
                            printf(".");
                        }
                        printf("%s", pth[i]);
                    }
                    printf("\n");
                }
            }
        }
    }
    return 0;
}
```

复杂度

n ⩽ 100，k ⩽ 100，随便怎么写都够快。操作 4 如果顺序查找元素，O(元素个数 × 成员数) 也完全没问题。

易错点提醒

1. 对齐的对象是"偏移量"而不是"上一个成员的结束位置"——其实两者等价，但一定要在放成员之前对齐，放完再加大小；
2. 结构体整体大小最后还要补齐到自身对齐要求的倍数，别漏；
3. 元素的起始地址也要对齐，而且内存是全局递增的，别每次从 0 开始；
4. 操作 4 里"落在洞中"要输出 `ERR`，这是样例 2 专门考察的点；
5. 输出路径时用 `.` 连接各段，注意不要在末尾多输出一个点。

第四题 P9755 [CSP-S 2023] 种树

题目描述

你是一个森林养护员，有一天，你接到了一个任务：在一片森林内的地块上种树，并养护至树木长到指定的高度。

森林的地图有 n 片地块，其中 1 号地块连接森林的入口。共有 n − 1 条道路连接这些地块，使得每片地块都能通过道路互相到达。最开始，每片地块上都没有树木。

你的目标是：在每片地块上均种植一棵树木，并使得 i 号地块上的树的高度生长到不低于 a_i 米。

你每天可以选择一个未种树且与某个已种树的地块直接邻接（即通过单条道路相连）的地块，种一棵高度为 0 米的树。如果所有地块均已种过树，则你当天不进行任何操作。特别地，第 1 天你只能在 1 号空地种树。

对每个地块而言，从该地块被种下树的当天开始，该地块上的树每天都会生长一定的高度。由于气候和土壤条件不同，在第 x 天，i 号地块上的树会长高 $\max(b_i + x \times c_i, 1)$ 米。注意这里的 x 是从整个任务的第一天，而非种下这棵树的第一天开始计算。

你想知道：最少需要多少天能够完成你的任务？

输入格式

第一行一个正整数 n。接下来 n 行，每行三个整数 a_i, b_i, c_i。接下来 n − 1 行，每行两个正整数 u_i, v_i，表示一条道路。

输出格式

输出一行一个正整数，表示完成任务所需的最少天数。

输入输出样例

输入 #1	输出 #1
4 12 1 1 2 4 -1 10 3 0 7 10 -2 1 2 1 3 3 4	5

【样例 1 解释】

- 第 1 天：在地块 1 种树，地块 1 的树木长高至 2 米。
- 第 2 天：在地块 3 种树，地块 1, 3 的树木分别长高至 5, 3 米。
- 第 3 天：在地块 4 种树，地块 1, 3, 4 的树木分别长高至 9, 6, 4 米。
- 第 4 天：在地块 2 种树，地块 1, 2, 3, 4 的树木分别长高至 14, 1, 9, 6 米。
- 第 5 天：地块 1, 2, 3, 4 的树木分别长高至 20, 2, 12, 7 米。

数据范围

对于所有测试数据有：$1 \leqslant n \leqslant 10^5$，$1 \leqslant a_i \leqslant 10^{18}$，$1 \leqslant b_i \leqslant 10^9$，$0 \leqslant |c_i| \leqslant 10^9$。保证存在方案能在 10^9 天内完成任务。

测试点编号	$n \leqslant$	特殊性质
1	20	A
2~4	20	无
5~6	500	A
7~8	10^5	A
9~10	10^5	B
11~13	10^5	C
14~16	10^5	D
17~20	10^5	无

特殊性质 A：对于所有 $1 \leqslant i \leqslant n$，均有 $c_i = 0$。

特殊性质 B：对于所有 $1 \leqslant i < n$，均有 $u_i = i, v_i = i + 1$（一条链）。

特殊性质 C：与任何地块直接相连的道路均不超过 2 条。

特殊性质 D：对于所有 $1 \leqslant i < n$，均有 $u_i = 1$（1 号是所有地块的中心）。

第一档：地块连成一条链（预估 10 分）

思路分析

先看特殊性质 B：地块 1-2-3-⋯-n 连成一条链。

链上最大的好处是：种植顺序是唯一的——第 1 天只能种 1 号，之后每天只能沿着链往后种一块，所以第 i 天种的就是第 i 号地块，没有任何选择的余地。

既然顺序固定，问题就退化成：每块地 i 从第 i 天开始长，各自算一算"最早第几天能长到 a_i"，取最大值就是答案。

怎么算？第 i 块地第 t 天种下，到第 D 天结束时的总高度是

> `sum( max(bᵢ + x × cᵢ, 1) )`，x 从 t 取到 D

这个和随着 D 增大只会越来越大，所以可以二分出最小的 D。

求和不能一天一天加（D 最大 10^9），要按等差数列的公式算，并且分三种情况：

- $c_i = 0$：每天长 b_i，总和就是 $(D - t + 1) \times b_i$；
- $c_i > 0$：越往后长得越快，而且 $b_i \geqslant 1$、$c_i > 0$、$x \geqslant 1$，所以 max 那一层不起作用，直接等差数列求和；
- $c_i < 0$：越往后长得越慢，慢到 $b_i + x \times c_i < 1$ 之后就只长 1。分界点是 $x \leqslant (b_i - 1) / (-c_i)$，分界点之前按等差数列算，之后每天加 1。

另外要注意溢出：a_i 最大 10^{18}，计算过程中很容易超出 `long long`。代码里用了"饱和加法 / 饱和乘法"：一旦超过一个很大的上限（取 4×10^{18}）就固定返回这个上限，反正我们只需要和 a_i 比大小。

这一档对应测试点 9~10，共 10 分。虽然分数不多，但"用公式算生长总量"这个函数（`calc`）是后面几档的基础，先写对很有价值。

本档代码（预估得分：10 分，对应测试点 9~10）

```
// P9755 [CSP-S 2023] 种树 —— 第一档：链（特殊性质 B，地块 1-2-...-n 连成一条链）
// 预估得分：10 分（对应测试点 9~10）
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int N = 100005;
const ll INF = 4000000000000000000LL;   // 一个"足够大"的数，防止溢出

int n;
ll a[N], b[N], c[N];

// 饱和加法：一旦超过 INF 就固定返回 INF（反正只要和目标高度比大小）
ll add(ll x, ll y)
{
    if (x >= INF - y)
    {
        return INF;
    }
    return x + y;
}

// 饱和乘法：超过 INF 就返回 INF；有非正数直接返回 0
ll mul(ll x, ll y)
{
    if (x <= 0 || y <= 0)
    {
        return 0;
    }
```

```
    if (x > INF / y)
    {
        return INF;
    }
    return x * y;
}

// 第 i 块地：第 l 天到第 r 天它都已经种下，这段时间一共长高多少
// 也就是 sum( max(b[i] + x * c[i], 1) )，x 从 l 到 r
ll calc(int i, ll l, ll r)
{
    if (l > r)
    {
        return 0;
    }
    ll bb = b[i];
    ll cc = c[i];
    ll cnt = r - l + 1;
    if (cc == 0)
    {
        return mul(cnt, bb);                    // 每天固定长 b[i]
    }
    if (cc > 0)
    {
        // 越往后长得越快，b[i] >= 1、c[i] > 0、x >= 1，max 不起作用
        // 和 = cnt * b[i] + c[i] * (l + r) * cnt / 2
        ll res = mul(cnt, bb);
        ll t = mul(cc, l + r);
        t = mul(t, cnt);
        res = add(res, t / 2);
        return res;
    }
    // cc < 0：越往后长得越慢，慢到 b[i] + x * c[i] < 1 之后每天只长 1
    // 分界点：b[i] + x * c[i] >= 1  <=>  x <= (b[i] - 1) / (-c[i])
    ll lim = (bb - 1) / (-cc);
    ll p = r;
    if (lim < p)
    {
        p = lim;
    }
    if (p < l)
    {
        return cnt;                             // 这段时间每天都只长 1
    }
    // 第 l 天到第 p 天：按等差数列求和
    ll c1 = p - l + 1;
    ll v1 = bb + l * cc;                        // 第 l 天长多少
    ll v2 = bb + p * cc;                        // 第 p 天长多少
    ll res = mul(c1, v1 + v2);
    res = res / 2;
    // 第 p + 1 天到第 r 天：每天都只长 1
    res = add(res, r - p);
    return res;
}

int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
```

```
    {
        scanf("%lld%lld%lld", &a[i], &b[i], &c[i]);
    }
    for (int i = 1; i < n; i++)
    {
        int u, v;
        scanf("%d%d", &u, &v);                    // 是链，边怎么连不影响：只能 1,2,...,n 这样种
    }
    // 链上顺序唯一：第 i 天种第 i 块地
    // 每块地各自算一算"最早第几天能长够"，取最大值就是答案
    ll ans = 0;
    for (int i = 1; i <= n; i++)
    {
        // 二分最小的 D，使第 i 天种下、到第 D 天结束时高度达标
        ll lo = i;
        ll hi = 1000000000LL;
        ll res = hi;
        while (lo <= hi)
        {
            ll mid = (lo + hi) / 2;
            if (calc(i, i, mid) >= a[i])
            {
                res = mid;
                hi = mid - 1;
            }
            else
            {
                lo = mid + 1;
            }
        }
        if (res > ans)
        {
            ans = res;
        }
    }
    printf("%lld\n", ans);
    return 0;
}
```

第二档：所有 $c_i = 0$，判定用 $O(n^2)$ 扫描（预估 15 分）

思路分析

$c_i = 0$（特殊性质 A）时，第 i 块地每天固定长 b_i，于是一下子好算很多：

- 它需要长 $g_i = \lceil a_i / b_i \rceil$ 天；
- 如果第 t 天种下，到第 D 天结束时一共长了 $(D - t + 1) \times b_i$，要求 $\geqslant a_i$；
- 也就是 $D - t + 1 \geqslant g_i$，即 $t \leqslant D - g_i + 1$。

记 $d_i = D - g_i + 1$，它的含义是：第 i 块地最晚必须在第 d_i 天种下。现在问题变成了一个纯粹的排班问题：

> 有 n 块地，每天种一棵（第 1 天只能种 1 号；之后只有"父亲地块已经种了"的地块才能种）。能不能安排一个顺序，使得每块地 i 都在第 d_i 天之前种下？

注意"每天种一棵、一共 n 棵"，所以前 n 天一定每天种一棵（中间空着不种只会更慢，不会更优），顺序就是 1~n 的一个排列，并且父亲必须排在儿子前面。

怎么判定？用一个很自然的贪心，而且是倒着安排：

> 从第 n 天往第 1 天安排。每次在"所有儿子都已经安排好了"的地块中，挑最不着急的（也就是 d_i 最大的）放到当前这一天。

直觉很好懂：越靠后的位置越"晚"，把晚的位置留给不着急的地块，靠前的位置就自然地留给了着急的地块。如果这样安排还是有某块地超过了它的期限，那就说明 D 天确实不够。（这个贪心是有严格保证的最优策略，它把"最晚完成时间的最大值"做到最小。）

实现上，这一档先用最朴素的 O(n) 扫描来找"最不着急的地块"，于是每次判定是 $O(n^2)$，配合二分答案一共 $O(n^2 \log V)$。$n \leqslant 500$ 时完全够用，对应测试点 1、5~6，共 15 分。

本档代码（预估得分：15 分，对应测试点 1、5~6）

```
// P9755 [CSP-S 2023] 种树 —— 第二档：所有 c[i] = 0（特殊性质 A）
// 预估得分：15 分（对应测试点 1、5~6）
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int N = 505;

int n;
ll a[N], b[N];                  // 这一档里 c 全是 0，所以不用存 c

// 建树用的邻接表
int head[N], to[N * 2], nxt[N * 2];
int ec;

void ade(int u, int v)
{
    ec++;
    to[ec] = v;
    nxt[ec] = head[u];
    head[u] = ec;
}

int fa[N];                      // 以 1 号地块为根之后，每块地的父亲
int cc[N];                      // 每块地有几个儿子
int que[N];                     // 广搜用的队列

void bfs()
{
    int fr = 0, re = 0;
    re++;
    que[re] = 1;
    fa[1] = 0;
    while (fr < re)
    {
        fr++;
        int u = que[fr];
```

```
        for (int e = head[u]; e != 0; e = nxt[e])
        {
            int v = to[e];
            if (v == fa[u])
            {
                continue;
            }
            fa[v] = u;
            cc[u]++;
            re++;
            que[re] = v;
        }
    }
}

ll dl[N];                       // dl[i]: 第 i 块地最晚第几天必须种下
int rem[N];                     // 还有几个儿子没安排
int done[N];                    // 是否已经安排过

// 判断: 给 D 天, 能不能完成任务
bool ok(ll D)
{
    // 第一步: c = 0 时每天长高都是 b[i], 需要长 g 天才能到 a[i]
    for (int i = 1; i <= n; i++)
    {
        ll g = (a[i] + b[i] - 1) / b[i];        // 就是 a[i] 除以 b[i] 向上取整
        dl[i] = D - g + 1;
        if (dl[i] < 1)
        {
            return false;                       // 就算第 1 天就种也来不及
        }
    }
    // 第二步: 倒着安排种植顺序
    for (int i = 1; i <= n; i++)
    {
        rem[i] = cc[i];
        done[i] = 0;
    }
    for (int pos = n; pos >= 1; pos--)
    {
        int best = -1;
        for (int i = 1; i <= n; i++)            // O(n) 扫描, n 大了会超时
        {
            if (done[i] == 0 && rem[i] == 0)
            {
                if (best == -1 || dl[i] > dl[best])
                {
                    best = i;
                }
            }
        }
        done[best] = 1;
        if (dl[best] < pos)
        {
            return false;                       // 轮到它时已经过了期限
        }
        if (fa[best] != 0)
        {
            rem[fa[best]]--;
```

```
        }
    }
    return true;
}

int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
    {
        ll t;
        scanf("%lld%lld%lld", &a[i], &b[i], &t);    // 第三个是 c，这一档里它是 0
    }
    for (int i = 1; i < n; i++)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        ade(u, v);
        ade(v, u);
    }
    bfs();
    // 二分答案：最少 n 天，最多 1e9 天
    ll L = n;
    ll R = 1000000000LL;
    while (L < R)
    {
        ll mid = (L + R) / 2;
        if (ok(mid))
        {
            R = mid;
        }
        else
        {
            L = mid + 1;
        }
    }
    printf("%lld\n", L);
    return 0;
}
```

第三档：所有地都直接连着 1 号（星形），顺序随便排（预估 15 分）

思路分析

再看特殊性质 D：所有边都是 (1, i)，也就是 1 号是中心，其他地块都是直接挂在 1 号上的"叶子"。

星形最大的好处是：除了第 1 天固定种 1 号之外，剩下的叶子想按什么顺序种都行（它们都直接和 1 号相邻）。这和链正好相反——链是顺序完全固定，星形是顺序完全自由。

顺序自由，问题就变成了经典的"排班"：对固定的总天数 D，每个叶子 i 有一个"最晚种植日" d_i，我们要给它们排个先后顺序，让每个叶子都在自己的期限之前种下。

这里 c_i 不一定是 0，所以"最晚种植日"不能像上一档那样直接算。但有一点没变：种得越晚，能长的天数越少，最终高度越低。所以对固定的 D，第 i 块地的 d_i 可以用第一档写好的 `calc` 二分出来：

- 先检查 `calc(i, 1, D) < a_i`，连第 1 天就种都来不及的话，D 天肯定不够；
- 否则二分最大的 t，使得 `calc(i, t, D) ≥ a_i`，这个 t 就是 d_i。

拿到所有 d_i 之后怎么安排？经典结论：按 d_i 从小到大种（最着急的先种）最优。第 1 天种 1 号，第 k 个种下的叶子在第 k + 1 天（k 从 1 到 n − 1），需要 $d \geqslant k + 1$。这个判断甚至不用真的排序，用桶数一数就行：

> 对每个 v，满足 $d_i \leqslant v$ 的叶子个数不能超过 v − 1（它们都得挤进第 2 ~ v 天，一共只有 v − 1 个位置）。

这一档对应测试点 14~16，共 15 分。它的代码里"二分 d_i"和"判定可行性"这两步，正是下一档正解的雏形。

本档代码（预估得分：15 分，对应测试点 14~16）（实际洛谷得分:95分,想想为什么？）

```cpp
// P9755 [CSP-S 2023] 种树 —— 第三档：星形（特殊性质 D，所有地都直接连着 1 号）
// 预估得分：15 分（对应测试点 14~16）
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int N = 100005;
const ll INF = 4000000000000000000LL;

int n;
ll a[N], b[N], c[N];

// 饱和加法、饱和乘法（和第一档完全相同）
ll add(ll x, ll y)
{
    if (x >= INF - y)
    {
        return INF;
    }
    return x + y;
}

ll mul(ll x, ll y)
{
    if (x <= 0 || y <= 0)
    {
        return 0;
    }
    if (x > INF / y)
    {
        return INF;
    }
    return x * y;
}

// 第 i 块地：第 l 天到第 r 天一共长高多少（和第一档完全相同）
ll calc(int i, ll l, ll r)
{
    if (l > r)
```

```
    {
        return 0;
    }
    ll bb = b[i];
    ll cc = c[i];
    ll cnt = r - l + 1;
    if (cc == 0)
    {
        return mul(cnt, bb);
    }
    if (cc > 0)
    {
        ll res = mul(cnt, bb);
        ll t = mul(cc, l + r);
        t = mul(t, cnt);
        res = add(res, t / 2);
        return res;
    }
    ll lim = (bb - 1) / (-cc);
    ll p = r;
    if (lim < p)
    {
        p = lim;
    }
    if (p < l)
    {
        return cnt;
    }
    ll c1 = p - l + 1;
    ll v1 = bb + l * cc;
    ll v2 = bb + p * cc;
    ll res = mul(c1, v1 + v2);
    res = res / 2;
    res = add(res, r - p);
    return res;
}

ll dl[N];                       // dl[i]: 第 i 块地最晚第几天必须种下
int buk[N];                     // buk[v]: dl 恰好等于 v 的叶子有几块

// 判断: 给 D 天，能不能完成任务
bool ok(ll D)
{
    // 第一步: 算出每块地最晚第几天必须种下
    for (int i = 1; i <= n; i++)
    {
        // 先花一次 calc 判断: 第 n 天（最晚）才种也来得及吗?
        // 来得及说明这块地不受限，dl 直接取 n，跳过二分（关键优化，省掉大量 calc）
        if (calc(i, n, D) >= a[i])
        {
            dl[i] = n;
            continue;
        }
        if (calc(i, 1, D) < a[i])
        {
            return false;                       // 连第 1 天就种都来不及
        }
        // 二分最大的 t，使第 t 天种下、到第 D 天结束时高度够（此时答案一定在 [1, n-1]）
        ll lo = 1, hi = n - 1, res = 1;
```

```
        while (lo <= hi)
        {
            ll md = (lo + hi) / 2;
            if (calc(i, md, D) >= a[i])
            {
                res = md;
                lo = md + 1;
            }
            else
            {
                hi = md - 1;
            }
        }
        dl[i] = res;
    }
    // 1 号地第 1 天就种，dl[1] >= 1 自动满足，不用额外检查
    // 第二步：叶子随便排序。第 k 个种下的叶子在第 k + 1 天（k 从 1 到 n-1）
    // 按 dl 从小到大安排最优，等价于：dl <= v 的叶子个数不能超过 v - 1
    for (int v = 0; v <= n; v++)
    {
        buk[v] = 0;
    }
    for (int i = 2; i <= n; i++)
    {
        buk[dl[i]]++;
    }
    int sum = 0;
    for (int v = 1; v <= n; v++)
    {
        sum = sum + buk[v];
        if (sum > v - 1)
        {
            return false;
        }
    }
    return true;
}

int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
    {
        scanf("%lld%lld%lld", &a[i], &b[i], &c[i]);
    }
    for (int i = 1; i < n; i++)
    {
        int u, v;
        scanf("%d%d", &u, &v);                  // 星形，边不用存（一定是 1 和别的点相连）
    }
    // 二分答案：最少 n 天，最多 1e9 天
    ll L = n;
    ll R = 1000000000LL;
    while (L < R)
    {
        ll mid = (L + R) / 2;
        if (ok(mid))
        {
            R = mid;
```

```
        }
        else
        {
            L = mid + 1;
        }
    }
    printf("%lld\n", L);
    return 0;
}
```

第四档：正解 —— 二分答案 + 倒序贪心 + 堆优化（预估 100 分）

思路分析

把第二档里"偷懒"的两个地方补上，就是正解。

补点一：处理 $c_i \neq 0$ 的一般情况。 这时"需要长多少天"不再是一个常数，因为每天长多少跟"是哪一天"有关。但和第三档一样，对固定的 D，第 i 块地"最晚第几天必须种下" d_i 可以用 `calc` 二分出来：先检查 `calc(i, 1, D) < a_i`，否则二分最大的 t 使得 `calc(i, t, D) ≥ a_i`。

补点二：把 O(n) 扫描换成堆。 第二档里每安排一天都要扫一遍全部地块找"最不着急的"，$n = 10^5$ 时不能接受。注意到我们每次要的是 d_i 最大的那个地块，这正是大根堆干的事：

- 一开始把所有"没有儿子"的地块（叶子）放进堆；
- 每安排好一个地块，就把它父亲的"未安排儿子数"减一，减到 0 时父亲就可以进堆了；
- 每次从堆顶取一个出来即可。

代码里手写了一个大根堆（只有入堆、出堆两个操作），比较规则按 d_i 从大到小。

整体流程：

1. 以 1 号地块为根把树建起来（广搜即可），记下每块地的父亲和儿子个数；
2. 二分答案 $D \in [n, 10^9]$；
3. 对每个 D 调用 `ok(D)`：算所有 d_i，再做一遍倒序贪心判定；
4. 输出最小的可行 D。

复杂度是 $O(n \log n \log V)$，$n = 10^5$、$\log V \approx 30$，完全可以通过。

本档代码（预估得分：100 分）

```
// P9755 [CSP-S 2023] 种树 —— 第四档：正解（二分答案 + 倒序贪心 + 堆优化）
// 预估得分：100 分
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int N = 100005;
const ll INF = 4000000000000000000LL;   // 一个"足够大"的数，防止加法乘法溢出
```

```
int n;
ll a[N], b[N], c[N];            // 每块地的目标高度 a、参数 b、c

// 建树用的邻接表
int head[N], to[N * 2], nxt[N * 2];
int ec;

void ade(int u, int v)
{
    ec++;
    to[ec] = v;
    nxt[ec] = head[u];
    head[u] = ec;
}

int fa[N];                      // 以 1 号地块为根之后，每块地的父亲
int cc[N];                      // 每块地有几个儿子
int que[N];                     // 广搜用的队列

void bfs()
{
    int fr = 0, re = 0;
    re++;
    que[re] = 1;
    fa[1] = 0;
    while (fr < re)
    {
        fr++;
        int u = que[fr];
        for (int e = head[u]; e != 0; e = nxt[e])
        {
            int v = to[e];
            if (v == fa[u])
            {
                continue;
            }
            fa[v] = u;
            cc[u]++;
            re++;
            que[re] = v;
        }
    }
}

// 饱和加法、饱和乘法：超过 INF 就固定返回 INF
ll add(ll x, ll y)
{
    if (x >= INF - y)
    {
        return INF;
    }
    return x + y;
}

ll mul(ll x, ll y)
{
    if (x <= 0 || y <= 0)
    {
        return 0;
```

```
    }
    if (x > INF / y)
    {
        return INF;
    }
    return x * y;
}

// 第 i 块地：第 l 天到第 r 天一共长高多少
ll calc(int i, ll l, ll r)
{
    if (l > r)
    {
        return 0;
    }
    ll bb = b[i];
    ll cc2 = c[i];
    ll cnt = r - l + 1;
    if (cc2 == 0)
    {
        return mul(cnt, bb);
    }
    if (cc2 > 0)
    {
        ll res = mul(cnt, bb);
        ll t = mul(cc2, l + r);
        t = mul(t, cnt);
        res = add(res, t / 2);
        return res;
    }
    // cc2 < 0：越往后长得越慢，慢到底之后每天只长 1
    ll lim = (bb - 1) / (-cc2);
    ll p = r;
    if (lim < p)
    {
        p = lim;
    }
    if (p < l)
    {
        return cnt;
    }
    ll c1 = p - l + 1;
    ll v1 = bb + l * cc2;
    ll v2 = bb + p * cc2;
    ll res = mul(c1, v1 + v2);
    res = res / 2;
    res = add(res, r - p);
    return res;
}

// 手写的大根堆：dl 越大越优先出堆
ll dl[N];                       // dl[i]：第 i 块地最晚第几天必须种下
int rem[N];                     // 还有几个儿子没安排
int hp[N];
int hs;

void push(int u)
{
    int i = hs + 1;
```

```
    hs++;
    hp[i] = u;
    while (i > 1)
    {
        int p = i / 2;
        if (dl[hp[p]] >= dl[hp[i]])
        {
            break;
        }
        int t = hp[p];
        hp[p] = hp[i];
        hp[i] = t;
        i = p;
    }
}

int pop()
{
    int res = hp[1];
    hp[1] = hp[hs];
    hs--;
    int i = 1;
    while (true)
    {
        int lc = i * 2;
        int rc = i * 2 + 1;
        int best = i;
        if (lc <= hs && dl[hp[lc]] > dl[hp[best]])
        {
            best = lc;
        }
        if (rc <= hs && dl[hp[rc]] > dl[hp[best]])
        {
            best = rc;
        }
        if (best == i)
        {
            break;
        }
        int t = hp[best];
        hp[best] = hp[i];
        hp[i] = t;
        i = best;
    }
    return res;
}

// 判断：给 D 天，能不能完成任务
bool ok(ll D)
{
    // 第一步：算出每块地最晚第几天必须种下
    for (int i = 1; i <= n; i++)
    {
        // 先花一次 calc 判断：第 n 天（最晚）才种也来得及吗？
        // 来得及说明这块地不受限，dl 直接取 n，跳过二分（关键优化，省掉大量 calc）
        if (calc(i, n, D) >= a[i])
        {
            dl[i] = n;
            continue;
```

```
        }
        if (calc(i, 1, D) < a[i])
        {
            return false;                       // 连第 1 天就种都来不及
        }
        // 二分最大的 t，使第 t 天种下、到第 D 天结束时高度够（此时答案一定在 [1, n-1]）
        ll lo = 1, hi = n - 1, res = 1;
        while (lo <= hi)
        {
            ll md = (lo + hi) / 2;
            if (calc(i, md, D) >= a[i])
            {
                res = md;
                lo = md + 1;
            }
            else
            {
                hi = md - 1;
            }
        }
        dl[i] = res;
    }
    // 第二步：倒着安排种植顺序
    // 从第 n 天往第 1 天，每次在"儿子都安排好了"的地块里挑 dl 最大的
    hs = 0;
    for (int i = 1; i <= n; i++)
    {
        rem[i] = cc[i];
        if (rem[i] == 0)
        {
            push(i);
        }
    }
    for (ll pos = n; pos >= 1; pos--)
    {
        int u = pop();
        if (dl[u] < pos)
        {
            return false;                       // 轮到它时已经超过最晚期限
        }
        if (fa[u] != 0)
        {
            rem[fa[u]]--;
            if (rem[fa[u]] == 0)
            {
                push(fa[u]);
            }
        }
    }
    return true;
}

int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
    {
        scanf("%lld%lld%lld", &a[i], &b[i], &c[i]);
    }
```

```
    for (int i = 1; i < n; i++)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        ade(u, v);
        ade(v, u);
    }
    bfs();
    // 答案最少也要 n 天（每天最多种一棵），最多 1e9 天，二分最小的可行天数
    ll L = n;
    ll R = 1000000000LL;
    while (L < R)
    {
        ll mid = (L + R) / 2;
        if (ok(mid))
        {
            R = mid;
        }
        else
        {
            L = mid + 1;
        }
    }
    printf("%lld\n", L);
    return 0;
}
```

复杂度

- 计算所有 d_i：O(n log n)（每块地二分一次，每次 `calc` 是 O(1)）；
- 倒序贪心：O(n log n)（堆）；
- 外层二分答案约 30 次，总复杂度 O(n log n log V)。

易错点提醒

1. 答案至少是 n 天：每天最多种一棵，一共 n 棵，所以二分下界从 n 开始；
2. 所有高度、天数、地址量级都可能超过 `int`，一律用 `long long`；
3. 防溢出：a_i 到 10^{18}，$b_i + x \times c_i$ 也能到 10^{18}，再乘上天数就会爆 `long long`。一定要用饱和加法 / 饱和乘法把中间结果截断；
4. $c_i < 0$ 时别忘了"长得慢到底之后每天只长 1"这一段，漏了会算少（或算成负数）；
5. 建树时一定要以 1 号地块为根，父亲必须先于儿子种下，这个方向别搞反；
6. 二分判定函数里每次要重置 `rem`（未安排儿子数）。

最后：考场上的建议

1. 先扫一遍数据范围表和特殊性质，把"白送的分"写稳：比如第四题的链、第三题的"没有结构体"、第二题的 $n \leqslant 8000$ 和"只有 a、b"、第一题的 $n = 1$。这些分数加起来往往能到 60~100 分，比硬刚正解划算得多。
2. 每一档都单独保存一份代码，别在正解代码上直接改，万一正解调不出来，还能交前一档的。
3. 大模拟题（第三题）按"功能"拆：先把对齐、布局算对，再处理访问路径，最后处理按地址反查。一层一层加，比一次性写完容易调试得多。
4. 注意数据范围里的量级（10^{18}），凡是涉及高度、地址、大小的变量，一律 `long long`，并在乘法前想清楚会不会溢出。